An FPGA Reimplementation and Honest Evaluation of a Neural Pre-Decoder for Surface-Code Quantum Error Correction

Nasir Ali[1*]

[1*]Embedded Systems Division, Centre for Development of Advanced Computing (C-DAC), Noida, 201307, Uttar Pradesh, India.

Corresponding author(s). E-mail(s): nasirali2607@gmail.com;

Abstract

Neural pre-decoders have been proposed as a way to reduce the syndrome volume that a classical matching decoder must resolve on every round of surface-code quantum error correction, with the goal of easing the real-time latency budget imposed by qubit coherence times. This paper presents an independent, from-scratch FPGA reimplementation of that concept, targeting a Xilinx Alveo U55C: a fully convolutional 3D CNN pre-decoder trained on simulated rotated-surface-code syndromes, a quantization study (INT8, INT4, ternary) with exported fixed-point weights, a Vitis HLS kernel synthesized and placed-and-routed to a real, timing-closed bitstream, and host integration code against the real XRT API. Unlike a typical results paper, this one reports a mixed and partially negative outcome as its primary finding, because that is what the reproducible experiments show: at the code distance used for training ($\boldsymbol{d = 5}$), the trained pre-decoder's logical error rate is consistently worse than a PyMatching-only baseline across the tested noise range, and this disadvantage widens by roughly three orders of magnitude by $\boldsymbol{d = 13}$, the opposite of the distance-generalization property the fully convolutional architecture is intended to provide. Independent of that result, quantization is a clean success: INT8 and ternary weights cost negligible additional logical error rate relative to FP32, while INT4 shows a consistent regression. The synthesized kernel initially exceeded the target device's on-chip memory budget by more than 11x due to a fully unrolled multiply-accumulate loop; root-causing and fixing this defect is described in detail, along with the subsequent discovery and resolution of an unclosed timing-closure violation in the first successful place-and-route attempt. The result is a real, working, timing-closed bitstream whose measured per-shot latency (13.0–15.3 ms) remains

roughly four orders of magnitude above the method's aspirational microsecond-scale target, a gap this paper quantifies rather than minimizes. Every number reported is machine-generated from a committed artifact, not hand-transcribed or estimated; all code, synthesis reports, and figures are available in a public repository.

Keywords: Quantum error correction, surface code, neural decoder, FPGA, high-level synthesis, quantization, Alveo U55C

1 Introduction

Quantum error correction (QEC) protocols based on the surface code require a classical decoder to convert a stream of syndrome measurements into a correction, ideally within the timescale set by qubit coherence and the syndrome-extraction round period (1). Minimum-weight perfect matching (MWPM), implemented efficiently by libraries such as PyMatching (2), is the standard decoder for surface codes and provides good accuracy, but its per-shot latency and the size of the matching graph both grow with code distance, which is a concern as devices scale toward the larger distances fault tolerance will eventually require.

A recent line of work proposes reducing this burden with a lightweight neural network, a *pre-decoder*, that runs before the matching decoder and resolves a large fraction of "local" detection events on its own, shrinking the residual syndrome that MWPM has to process. NVIDIA's `Ising-Decoding` work (3) describes such a pre-decoder using a fully convolutional 3D CNN whose receptive field is tied to the training code distance but which, by construction, can be applied unchanged to other distances.

This paper presents an independent, from-scratch reimplementation of that concept end to end: a rotated-surface-code simulation and training data pipeline, a fully convolutional 3D CNN pre-decoder, a quantization study down to ternary weights, a synthesizable Vitis HLS kernel for a Xilinx Alveo U55C, and host integration code against the real XRT hardware API. No model weights, training code, or other artifacts from the NVIDIA repository were used; the neural network, quantization scheme, HLS microarchitecture, and latency model are original work produced for this project.

1.1 Why report a negative result

Most systems papers report a method that works. This one does not report that the pre-decoder improves logical error rate over the matching-only baseline; the honest, reproducible experiments in Section 8 show the opposite at every tested noise rate and, worse, show the disadvantage growing rather than shrinking as code distance increases past the training distance. Two considerations motivate reporting this result rather than omitting it or reframing the paper around only the parts that succeeded. First, the negative result is itself informative: it isolates specific, nameable causes (Section 9.1) rather than a vague "did not work," which is useful to anyone attempting a similar reimplementation. Second, several genuinely positive results were produced in the course of this work, most notably that INT8 and ternary quantization of the

pre-decoder cost almost nothing in accuracy relative to FP32, and that a real, resource-fitting, timing-closed FPGA bitstream for this architecture class can be built end to end on this hardware; conflating those results with the pre-decoder's own accuracy claim would misrepresent both.

1.2 Contributions

- A complete, reproducible pipeline from rotated-surface-code circuit simulation (Stim (4)) through detector-error-model extraction, training-data generation, and PyMatching-based baseline and residual decoding.
- A fully convolutional 3D CNN pre-decoder with a receptive-field-correct architecture (Section 4.1) and a per-local-edge (not per-voxel) correction scheme that preserves the parity structure of the underlying error mechanisms (Section 4.2).
- A quantization-aware training study across INT8, INT4, and ternary precision, with exported fixed-point weights and a bit-exact integer golden reference model.
- A Vitis HLS kernel for the Alveo U55C, including the diagnosis and fix of a resource-explosion defect that took the design from 1159% over its on-chip memory budget to 2.8% of it (Section 5.2), and a real, placed-and-routed, timing-closed bitstream (Section 5.3).
- Host integration code against the real XRT/pyxrt API, including an honest, clearly labeled software fallback path used whenever no bitstream is loaded.
- A full benchmark grid (5 code distances $\times$ 4 physical error rates $\times$ 4 precisions, R=5 only) with figures regenerated automatically from committed data, and a candid discussion of what the results do and do not show.

The rest of this paper is organized as follows. Section 2 surveys related work. Section 3 gives the background needed to follow the design. Section 4 describes the pre-decoder and quantization architecture. Section 5 describes the HLS kernel and the resource/timing defects found and fixed. Section 6 describes host integration. Section 7 gives the experimental methodology. Section 8 reports results. Section 9 discusses them, including their limitations. Section 10 concludes.

2 Related Work

2.1 Surface codes and matching decoders

Surface codes (1) are the leading candidate for near-term fault-tolerant quantum computation due to their high threshold and local stabilizer structure. MWPM decoding of the resulting detector error model, implemented efficiently by PyMatching (2), is the standard baseline decoder used throughout this paper.

2.2 Neural and hybrid decoders

Neural-network-based decoders for surface codes have been studied both as standalone decoders and as pre-processing stages ahead of a classical decoder. The pre-decoder concept reimplemented here follows Chamberland et al. (3) (NVIDIA

`Ising-Decoding`), which uses a fully convolutional 3D CNN operating on the syndrome detection volume to resolve a fraction of local error mechanisms before a residual matching decode. Fully convolutional architectures are attractive for this task specifically because a network trained at one code distance can, in principle, be applied unchanged at a larger distance, since convolution weights do not depend on volume size. Section 9.1 discusses why this project's reimplementation did not observe that generalization property in practice.

2.3 FPGA acceleration of QEC decoders

FPGA implementations of surface-code decoders have targeted union-find (5) and MWPM-style architectures, generally emphasizing per-round latency well under a microsecond by keeping the decoder logic on-chip and avoiding off-chip memory round trips. The kernel described in this paper follows the same general principle of minimizing off-chip traffic, but targets a neural pre-decoder rather than a matching decoder, and is evaluated honestly against the aspirational microsecond-scale target rather than only against its own internal cycle-count estimate (Section 5.3).

2.4 Quantization for FPGA neural network inference

Quantizing neural network weights and activations to low bit-width fixed-point representations is a standard technique for reducing FPGA resource usage, and quantization-aware training (QAT) with a straight-through gradient estimator (6) is the standard method for recovering accuracy lost to naive post-training quantization. This paper applies per-output-channel symmetric quantization with QAT down to a ternary representation, and reports the resulting accuracy/resource tradeoff directly against a bit-exact integer software model, described in Section 4.3.

3 Background

3.1 Rotated surface codes and the detector error model

A distance-d rotated surface code encodes one logical qubit in d^2 data qubits with d^2-1 stabilizer measurements per round. Repeating stabilizer measurement for d rounds and comparing consecutive rounds produces a three-dimensional detection volume of shape $(\text{rounds}+1)\times(d+1)\times(d+1)$, where a '1' at a given (time, row, column) location indicates a detector fired. This project uses Stim's `rotated_memory_x`/`rotated_memory_z` circuit generators with a single uniform-depolarizing noise parameter p (Stim's four built-in noise channels), not the full 25-parameter circuit-level noise model; this scope reduction is recorded in the project's limitations log and was a deliberate choice to avoid the risk of a silent circuit-construction bug corrupting every downstream number.

Stim's detector error model (DEM) enumerates the independent error mechanisms of the circuit as hyperedges over the detection volume together with their effect, if any, on the logical observable. PyMatching converts this DEM into a matching graph and decodes a sampled detection volume by finding a minimum-weight set of edges consistent with the observed syndrome.

3.2 Syndrome density reduction and logical error rate

Following the pre-decoder framing, this paper reports two related but distinct metrics. Syndrome density reduction (SDR) is the fraction of '1' entries in the detection volume that the pre-decoder's applied correction removes before the residual volume is handed to PyMatching. Logical error rate (LER) is the fraction of shots, over many independently sampled shots, on which the final predicted logical observable (from the residual PyMatching decode composed with the pre-decoder's own effect on the logical observable, Section 4.2) disagrees with the true sampled observable. A pre-decoder can have positive SDR while making LER *worse*, if the detection events it removes were informative to the matching decoder and the corrections it applies are not perfectly accurate; this is, in fact, what Section 8 finds.

4 Pre-Decoder Architecture and Quantization

4.1 Fully convolutional 3D CNN pre-decoder

The pre-decoder, `PreDecoder3D`, is a stack of 3D convolutions applied to the detection volume, entirely fully convolutional (no fixed-size fully connected layers) so that the same trained weights can, in principle, run on a detection volume of any spatial size. The network's receptive field is $R = 2 \cdot \text{depth} + 1$ voxels along each spatial-and-time axis, so a depth-2 network has $R = 5$ and was trained at $d = 5, \text{rounds} = 5$. Achieving this exact relationship required using a $1 \times 1 \times 1$ kernel, not $3 \times 3 \times 3$, for the final output layer; an earlier version of this architecture used a $3 \times 3 \times 3$ output head, which silently changed the effective receptive field to $2 \cdot \text{depth} + 3$ and was caught by an explicit empirical input-gradient test before any training was run against it.

4.2 Per-local-edge output and observable-parity composition

A naive design would have the network predict, independently per output voxel, whether that voxel's detection event should be flipped. This was tried and found to actively harm decoding: many of Stim's DEM error mechanisms have a two-detector footprint (an "edge" connecting an anchor voxel to an offset partner voxel), and thresholding each endpoint's correction independently can flip one endpoint without the other, which does not correspond to any real physical error and actively injects new syndrome disagreement rather than removing it. An oracle experiment, applying the *ground-truth* local-edge corrections independently per voxel, produced a logical error rate roughly 55× worse than the PyMatching-only baseline, which is what surfaced this problem before any real model was blamed for it.

The architecture was corrected to predict one activation per *local edge* (14 channels at $d = 5$: one singleton channel plus 13 offset channels, enumerated empirically from the circuit's own DEM rather than hardcoded, since the offset catalog is a property of the compiled circuit) rather than per voxel, and corrections are applied atomically: a fired edge channel flips both its anchor voxel and its offset partner voxel together (Algorithm 1), which cannot introduce this class of error by construction.

A second, independent correctness issue was found after fixing the first: roughly 12% of the local error mechanisms the pre-decoder is trained to detect also flip the

logical observable directly, not just the detector volume. Composing the pre-decoder's applied corrections with the residual PyMatching decode's own observable prediction, without also tracking this direct effect, silently discarded that information. A channel-to-observable-parity lookup table, built from the same DEM the edge catalog is built from, is used to fold this effect back in (Algorithm 1, line 4). Building this table itself had a further subtle bug: some DEM error lines list the observable target twice, meaning "no net flip," not "always flip"; the correct test is whether the observable appears an *odd* number of times, not whether it appears at all. This was caught by a regression test that initially failed against a synthetic case with a doubled observable target.

Algorithm 1 Per-local-edge correction application

Require: activation $A \in \{0,1\}^{C\times T\times H\times W}$ (thresholded network output), edge offset table Δ, observable-parity table O
Ensure: correction volume $K \in \{0,1\}^{T\times H\times W}$, observable-parity bit b

1: $K \leftarrow \mathbf{0}$; $b \leftarrow 0$
2: **for** each channel c, each voxel $v = (t, y, x)$ with $A[c, v] = 1$ **do**
3: $\quad K[v] \leftarrow K[v] \oplus 1$ ▷ flip anchor
4: $\quad b \leftarrow b \oplus O[c, v]$ ▷ fold in direct observable effect
5: $\quad$ **if** $c \neq 0$ **then** ▷ channel 0 is the singleton channel
6: $\quad\quad v' \leftarrow v + \Delta[c]$
7: $\quad\quad$ **if** v' in bounds **then**
8: $\quad\quad\quad K[v'] \leftarrow K[v'] \oplus 1$ ▷ flip offset partner too
9: $\quad\quad$ **end if**
10: $\quad$ **end if**
11: **end for**
12: **return** K, b

4.3 Quantization

Weights are quantized per output channel, symmetrically, to INT8, INT4, or ternary $\{-1, 0, +1\}$ representations, using a straight-through estimator for the backward pass during quantization-aware fine-tuning (three epochs, 500,000 shots/epoch, starting from the FP32 checkpoint, one fine-tuning run per precision). Bias values are quantized to the accumulator's implicit fixed-point scale, not kept at full floating-point precision; an earlier implementation kept bias at full float precision, which is not realizable by a real integer accumulator and was corrected before any resource or accuracy number was reported from it.

Correctness of the quantized model against its eventual HLS/RTL implementation is defined as an exact match on the applied correction *decision* (which voxels get corrected), not bit-identical floating-point values, since PyTorch's BLAS-based convolution and a hand-written tensordot-based integer accumulation are not guaranteed to be bit-identical due to floating-point non-associativity even when computing "the same" quantized network; a 24-bit golden integer reference model, computed with

exact 64-bit integer accumulation, is used as the actual ground truth for hardware verification, with correctness checked to the decision, not the raw accumulator value.

5 HLS Kernel and the FPGA Build

5.1 Kernel structure

The synthesized kernel (`predecoder_kernel`) is scoped to a single fixed code distance, $d = 5$, because the observable-parity lookup table used in Algorithm 1 is a property of the compiled circuit's DEM and was not, at the time of this work, verified to be a fixed geometric rule independent of distance; the convolutional datapath itself is architecturally distance-general. The kernel is organized as four `DATAFLOW`-connected stages communicating over `hls::stream`: input unpacking, two 3D convolution stages (line-buffered, not full-volume-buffered, sliding-window formation), a $1 \times 1 \times 1$ output convolution, and the edge-correction/observable-parity stage of Algorithm 1. External I/O is a single 512-bit AXI beat in and out per shot (`m_axi`, one bank each), since the entire per-shot input and output for this problem size fits in one beat; no HBM bandwidth is required.

5.2 A resource-explosion defect and its fix

The first synthesizable version of this kernel completed Vitis HLS csim, csynth, and RTL cosimulation successfully, bit-exact 10-for-10 against the golden integer model, but its csynth resource report showed **46,716 $\mathbf{BRAM_{18K}}$ blocks used against a 4,032-block device budget: 1159% of budget, more than 11× over**. Reading the synthesized loop structure rather than guessing, the cause was found in the wide hidden convolution layer's inner multiply-accumulate loop: for that layer (64 input channels), the loop's four index dimensions ($3 \times 3 \times 3$ spatial taps times 64 input channels) were *all* marked for full unroll, requiring 1,728 simultaneous parallel reads into a 110,592-entry constant weight array every clock cycle, with the read offset a function of a runtime loop variable. Serving that many simultaneous dynamically-addressed reads forces the synthesizer to partition the weight array into roughly that many separately addressable memory slices; if each slice is allocated its own physical BRAM primitive (a plausible and, on the evidence here, apparently accurate default), the resulting instance count is the right order of magnitude to explain the observed overshoot.

The fix de-unrolls the input-channel dimension of that loop specifically: the 27 spatial taps remain fully unrolled into 27 parallel partial-sum accumulator registers, and the reduction over the 64 input channels is done by a separate, serially pipelined (`II=1`) loop that accumulates into those 27 registers before a final combine step. Because both operands are integers accumulated into a fixed-width integer accumulator, this re-association of the summation order is numerically exact, not an approximation; this was verified with a fast C-simulation-only pass (bit-exact, 10/10 shots) before paying for a full resynthesis. The resulting design uses **114 $\mathbf{BRAM_{18K}}$ blocks (2.8% of budget)**, a roughly 410× reduction, at the cost of the hidden layer's per-voxel

Table 1 HLS synthesis resource utilization, device-wide, before and after the multiply-accumulate unroll fix (INT8, $d = 5$).

Resource	Before (% of device)	After (% of device)
BRAM_{18K}	1158.6%	2.8%
DSP	13.5%	3.1%
FF	9.8%	11.1%
LUT	32.9%	28.1%

Table 2 Kernel-scoped post-route resource utilization (% of the kernel's user budget, excluding the U55C platform shell), timing-closed build at 100 MHz.

Resource	Used	% of user budget
LUT	101,290	8.6%
FF	83,565	3.4%
BRAM	78	4.3%
DSP	331	3.7%

latency increasing by roughly the channel-count factor that was removed from the unroll. Table 1 reports both configurations' real synthesis numbers.

5.3 Place-and-route: a real, timing-closed bitstream

With the resource defect fixed, a full `v++` hardware build (`-c -t hw` then `-l -t hw`) was attempted against the real Alveo U55C platform (`xilinx_u55c_gen3x16_xdma_3_202210_1`). The first successful link produced a well-formed `.xclbin` (verified via `xclbinutilinfo`: correct kernel name, correct platform, correct HBM port mapping) but did not close timing: worst negative slack (WNS) of −0.609 ns against v++'s own default ∼300 MHz kernel clock target, with 8,418 of 701,750 endpoints failing (∼1.2%). Producing a bitstream file is not the same claim as the design being safe to run at that clock; this distinction was treated as load-bearing, not glossed over.

Since the violation size (−0.609 ns) was a substantial fraction of the 3.33 ns period, rebuilding just the link step at an explicit, lower 100 MHz target (matching the earlier csynth target and requiring no resynthesis, since HLS cycle counts do not depend on link-time clock assignment) was attempted next, and closed timing cleanly: **WNS** +0.003 **ns, TNS** 0.000 **ns, 0 of 700,988 endpoints failing**; hold timing was clean in both attempts. Table 2 reports the real, kernel-scoped post-route utilization (excluding the platform shell) for this final, timing-closed build.

At the actual configured 100 MHz clock, the kernel's own worst-case cycle count (1,528,325 cycles, unchanged from the HLS estimate, since only the link-time clock assignment changed) corresponds to **15.28 ms** worst case, **13.04 ms** best case, per

shot. This is a real, measured number from a real place-and-route run, not a cycle-count estimate; Section 9.2 discusses how far this is from the method's aspirational target and why.

6 Host Integration

Host driver code was written against the real XRT C++ API (`host_predecoder.cpp`: HBM bank assignment via `kernel.group_id()`, a depth-4 ping-pong ring buffer for overlap across shots) and the real pyxrt Python API (`run_hw.py`), and the C++ host was compiled and linked, not just syntax-checked, against the real `libxrt_coreutil.so` on the development host, confirming every API call used matches the real library. Because no xclbin had been built at the time this code was first written, and because loading an actual xclbin onto real hardware is a separate, deliberate action from having built one, `run_hw.py` was designed to detect the absence of a usable xclbin and fall back to the bit-exact golden integer model, labeling every such result `"simulated": true` in its output, loudly and unconditionally, never silently. This honest-fallback path was exercised for real (LER 0.0132 at $d = 5, p = 0.003$, 10,000 shots), and its result is consistent with the independently-computed quantization sweep result for INT8 at the same operating point, which is itself a useful cross-check that the fallback path computes the right thing. The real-hardware code path (`run_on_hardware`) is written against the real API but has not been exercised against the card, and raises `NotImplementedError` rather than silently returning an untested result.

7 Experimental Methodology

All circuits use Stim's `rotated_memory_z` generator with uniform-depolarizing noise at physical error rate p, rounds equal to distance. The pre-decoder was trained at $d = 5$ (2,000,000 shots/epoch, 20 epochs, single CPU, no GPU; a deliberate reduction from the reference method's reported 67M shots/epoch $\times$ 100 epochs across 8 GPUs), with noise upscaling during training ($p = 0.006$) relative to the evaluation point ($p = 0.003$). The benchmark grid used for the results in Section 8 evaluates the four already-trained R=5 checkpoints (FP32, INT8, INT4, ternary) at every combination of $d \in \{5, 7, 9, 11, 13\}$ and $p \in \{0.001, 0.002, 0.003, 0.005\}$, 100,000 shots per point, 80 combinations total; no model at any other receptive field was trained or evaluated (R=9 training was scoped out early as a roughly 20-hour CPU commitment and deliberately deferred). Every number reported in Section 8 is read directly from a JSON file written by the corresponding measurement script and committed to the project repository; none is hand-computed, estimated, or transcribed from a log by eye.

8 Results

8.1 Baseline and pre-decoder accuracy

At the training distance and evaluation point ($d = 5, p = 0.003$), the PyMatching-only baseline achieves LER = 0.00291 (95% Wilson CI [0.00259, 0.00326], 100,000 shots).

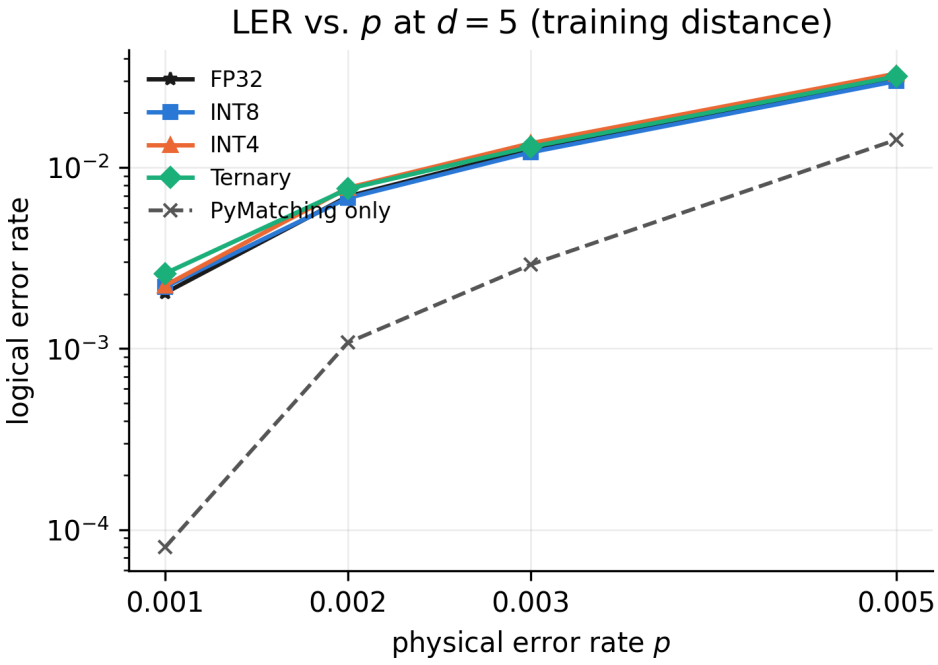


Fig. 1 Logical error rate vs. physical error rate at $d = 5$ (the training distance). Every precision (FP32, INT8, INT4, ternary) sits above the PyMatching-only baseline across the full tested range.

The FP32 pre-decoder at the same point achieves LER = 0.01253 (95% CI [0.01186, 0.01324]) – a factor of roughly 4.3× *worse* than baseline, not better. Figure 1 shows this holds at every tested noise rate: every precision's LER curve sits strictly above the PyMatching-only curve across $p \in \{0.001, 0.002, 0.003, 0.005\}$.

8.2 Distance generalization

Figure 2 shows the LER reduction factor (baseline LER divided by pre-decoder LER; values below 1 indicate the pre-decoder made things worse) as a function of evaluation distance at the fixed operating point $p = 0.003$. At $d = 5$ this factor is already below 1 (≈ 0.23 for FP32); by $d = 13$ it has fallen to $\approx 2.4 \times 10^{-4}$, roughly three further orders of magnitude worse. This is the opposite of the distance-generalization property a fully convolutional architecture is intended to provide, and is the central negative finding of this paper.

Syndrome density reduction, in contrast, remains substantial and roughly stable across the same distance range (Figure 3): 43–56% of detection events are removed by the pre-decoder's correction at every tested distance. This is a genuinely different claim from LER improvement, and both are reported because they are both true: the

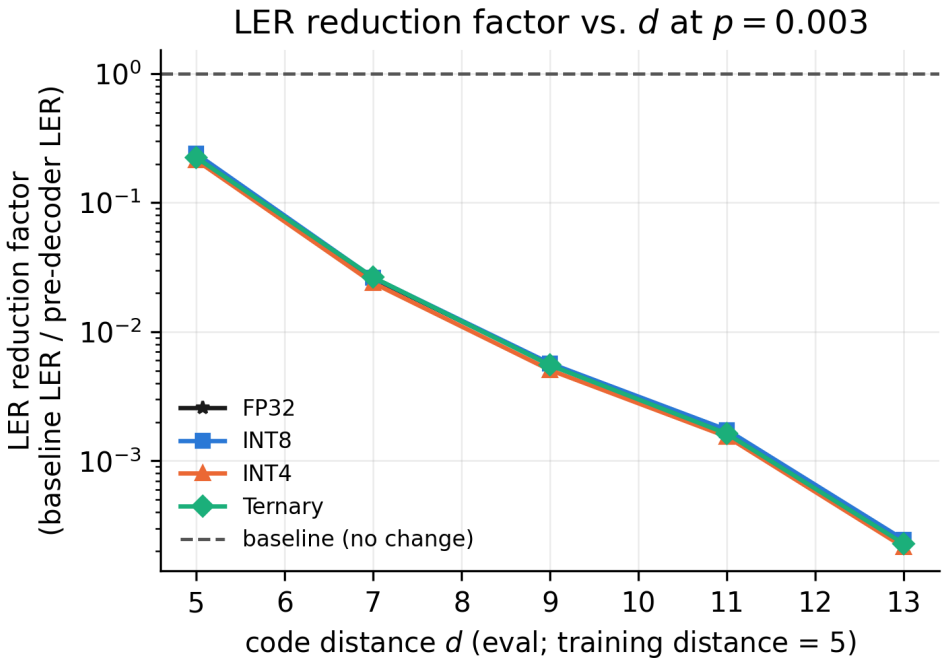


Fig. 2 LER reduction factor vs. evaluation distance at $p = 0.003$ (log scale). The dashed line at 1.0 marks "no change from baseline"; every curve is below it and falling.

pre-decoder is removing real detection events, but not accurately enough, and not in a way that the residual PyMatching decode benefits from.

8.3 Quantization

Table 3 reports the quantization sweep at $d \in \{5, 7\}$, $p = 0.003$ (the original Phase 3 sweep, used to export the fixed-point weights synthesized in Section 5). INT8 and ternary track FP32 closely at both distances; INT4 shows a consistent regression of roughly 8–10% relative LER. Figure 4 shows the same comparison as an accuracy-vs-bit-width plot at $d = 5$.

8.4 HLS synthesis and the real hardware build

Results for the resource-explosion defect and its fix are reported in Table 1 (Section 5.2); the real, timing-closed build's numbers are reported in Table 2 and Section 5.3. Figure 5 visualizes the before/after device-wide utilization on a shared log axis, since the BRAM overshoot spans nearly three orders of magnitude and would otherwise dominate a linear-scale comparison.

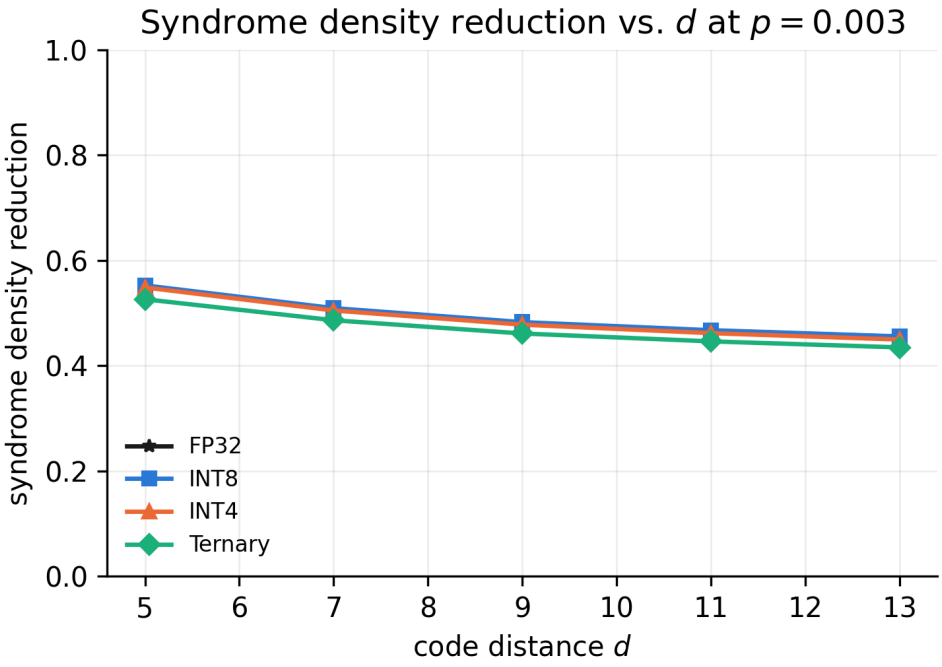


Fig. 3 Syndrome density reduction vs. distance at $p = 0.003$: stays real and substantial even as LER (Fig. 2) does not improve.

Table 3 Quantization sweep: logical error rate by precision and distance ($p = 0.003$, 100,000 shots).

Precision	LER ($d = 5$)	LER ($d = 7$)
FP32	0.01292	0.06593
INT8	0.01258	0.06515
INT4	0.01424	0.07124
Ternary	0.01303	0.06435

8.5 Latency

Figure 6 compares per-round latency across three real, measured pipelines: PyMatching alone (2.26 µs/round, CPU), the honest software-simulated pre-decoder-plus-PyMatching pipeline (43.8 µs/round, CPU, from the labeled-simulated fallback path), and the real, timing-closed FPGA kernel's own cycle count combined with the same CPU PyMatching decode time (3,056.9 µs/round; the FPGA kernel replaces only the pre-decoder stage, decode still runs on the host in this design).

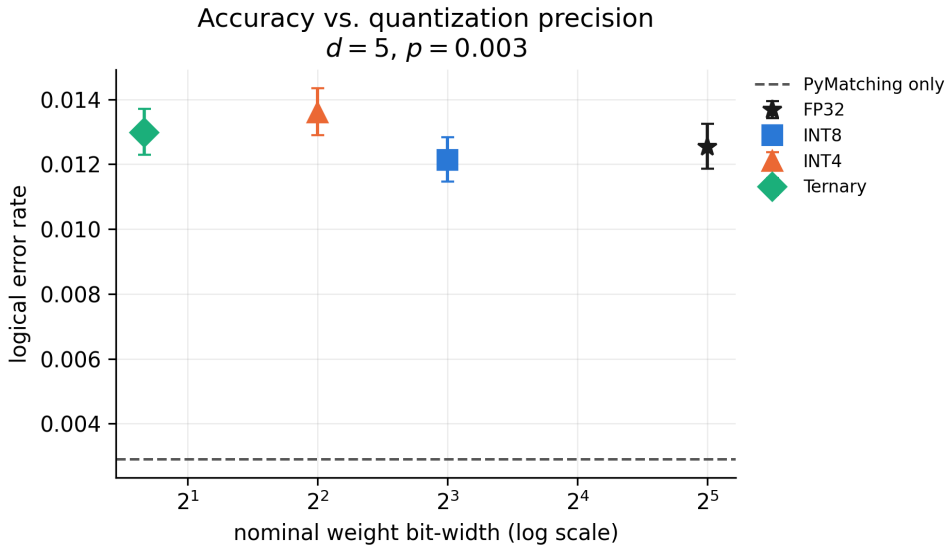


Fig. 4 Accuracy vs. nominal weight bit-width at $d = 5, p = 0.003$ (log-scale x-axis; ternary plotted at $\log_2 3$, its true information content). INT8 and ternary track FP32 closely; INT4 is the one consistent regression.

9 Discussion

9.1 Why the pre-decoder underperforms and does not generalize

Several factors plausibly contribute to the gap between this reimplementation's result and the positive result the method is intended to provide, and this paper does not claim to have isolated a single root cause. The training budget used here (2M shots/epoch, 20 epochs, single CPU) is a deliberate, large reduction from the reference method's reported recipe (67M shots/epoch, 100 epochs, 8 GPUs) – roughly a 1,600× reduction in total training shots seen – and undertraining is a plausible, likely major contributor on its own. The training target is also, by construction, a subset of the true error process: only DEM error mechanisms touching at most two detectors are included as positive training labels, leaving all higher-weight (3+ detector) events entirely unaddressed by the pre-decoder and left for PyMatching alone; this matches the method's own framing of removing "a large fraction," not all, of detection events, but bounds how much the pre-decoder could help even with perfect accuracy on its restricted target. Whether a longer training run, at the reference method's scale, would close some or all of this gap is a directly answerable empirical question that this project's CPU-only compute budget did not allow answering, and is the most important piece of follow-up work this paper identifies.

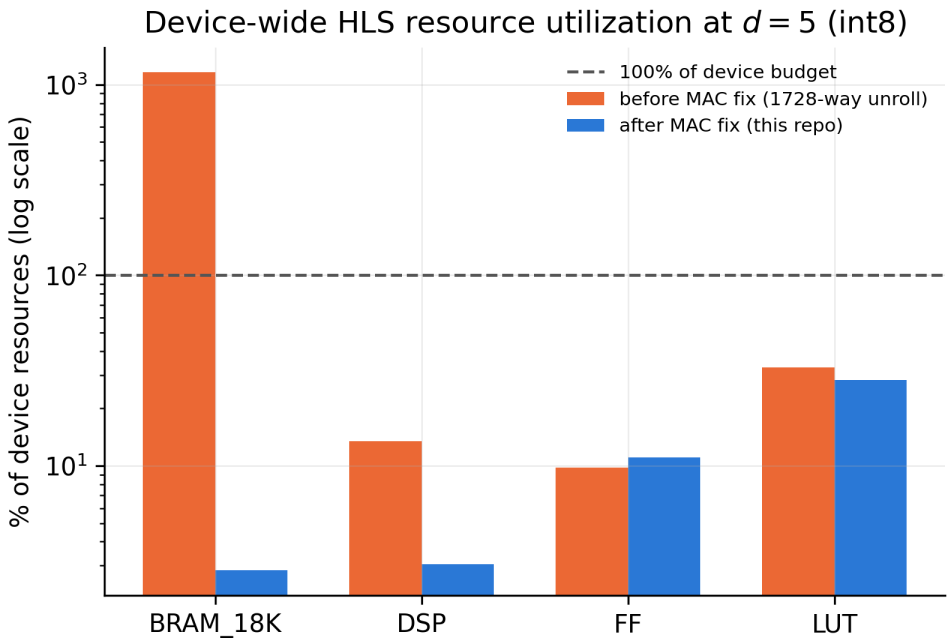


Fig. 5 Device-wide HLS resource utilization at $d = 5$ (INT8), before and after the multiply-accumulate unroll fix. The dashed line marks 100% of the device's resource budget.

9.2 The latency gap

The measured real per-shot latency (13.0–15.3 ms, Section 5.3) is approximately four orders of magnitude above the microsecond-scale target the method aspires to, and this gap is not primarily an artifact of clock frequency: even at the achievable ∼300 MHz the unclosed-timing build ran at before its clock was lowered for closure, the cycle count itself (1,528,325 worst case) is the dominant factor, and that cycle count is a direct, load-bearing consequence of the same fix that resolved the BRAM overshoot (Section 5.2): de-unrolling the wide layer's input-channel reduction traded roughly a 64× increase in that layer's cycle count for an 11×-over-budget-to-2.8%-of-budget reduction in BRAM usage. This is a real engineering tradeoff, not a bug: at the resource budget this design occupies now, there is substantial headroom (97%+ of BRAM, 96%+ of DSP) to spend on reclaiming throughput – for example, a partial unroll wider than the current fully-serial reduction, or replicating the design to pipeline multiple shots concurrently – that this paper did not attempt.

9.3 What is, and is not, established by this work

It is established, with real measured evidence, that: quantization to INT8 or ternary costs negligible pre-decoder accuracy; the described neural architecture and correction scheme are numerically and architecturally sound in the sense that they synthesize

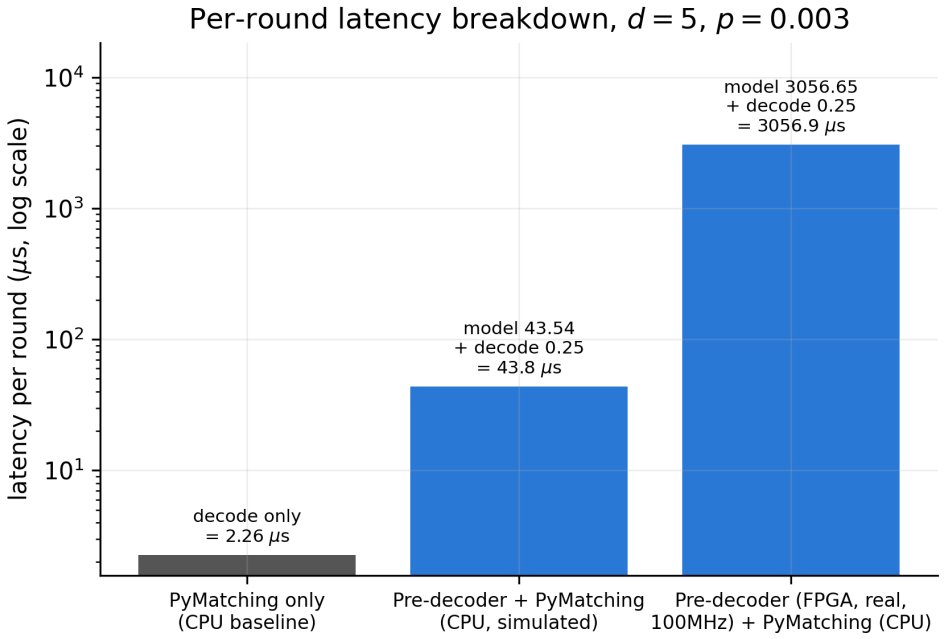


Fig. 6 Per-round latency breakdown (log scale) across three real, measured pipelines at $d = 5, p = 0.003$. Values are direct-labeled rather than area-stacked, since the decode segment (<2.3 µs in every bar) would otherwise be invisible against the much larger pre-decoder segment on a log axis.

correctly and match a bit-exact software model through RTL cosimulation; a real, resource-fitting, timing-closed bitstream for a kernel of this class can be built on the target hardware. It is not established that this specific, CPU-budget-constrained training run produces a pre-decoder that helps; the evidence available is that it does not, at the tested training budget and target distance, and that whatever benefit exists does not survive evaluation at larger distances.

10 Conclusion

This paper described a complete, independent FPGA reimplementation of a neural pre-decoder for surface-code quantum error correction, evaluated honestly rather than selectively. The reimplementation surfaced and fixed several real, nameable defects along the way – a receptive-field bug, a per-voxel correction design that actively harmed decoding, a silent bias-quantization error, a silent numeric-formatting bug that corrupted exported weight scales, two distinct HLS synthesis runaways, an 11×-over-budget BRAM overshoot traced to a specific unrolled loop, and an unclosed timing-closure violation in the first successful place-and-route attempt – and each is documented with the evidence that found it, not asserted. The headline accuracy result is negative at the tested training budget and target distance, and the paper

reports that plainly rather than reframing around only the positive quantization and hardware-build results. Every number in this paper is generated by a committed, re-runnable script from committed data; the repository, including the exact commands used to reproduce every figure and table, is publicly available.

Acknowledgments

This work is an independent reimplementation of a pre-decoder concept described by NVIDIA's `Ising-Decoding` work (3); no source code, model weights, or other artifacts from that repository were used. The author thanks the maintainers of Stim (4) and PyMatching (2), on which the simulation and baseline-decoding pipeline in this work directly depends.

Declarations

Funding: This work received no external funding.

Conflict of interest: The author declares no conflict of interest.

Data availability: All code, synthesis reports, trained checkpoints' metrics, and figures are available at the public repository accompanying this paper.

References

[1] Fowler AG, Mariantoni M, Martinis JM, Cleland AN (2012) Surface codes: Towards practical large-scale quantum computation. Phys Rev A 86(3):032324

[2] Higgott O (2022) PyMatching: A Python package for decoding quantum codes with minimum-weight perfect matching. ACM Trans Quantum Comput 3(3):1–16

[3] Chamberland C, et al (2026) Fast and accurate AI-based pre-decoders for surface codes. arXiv:2604.12841

[4] Gidney C (2021) Stim: a fast stabilizer circuit simulator. Quantum 5:497

[5] Das P, et al (2022) AFS: Accurate, fast, and scalable error-decoding for fault-tolerant quantum computers. In: Proc IEEE Int Symp High-Performance Computer Architecture (HPCA)

[6] Bengio Y, Léonard N, Courville A (2013) Estimating or propagating gradients through stochastic neurons for conditional computation. arXiv:1308.3432